NIXL Cluster Topology Discovery

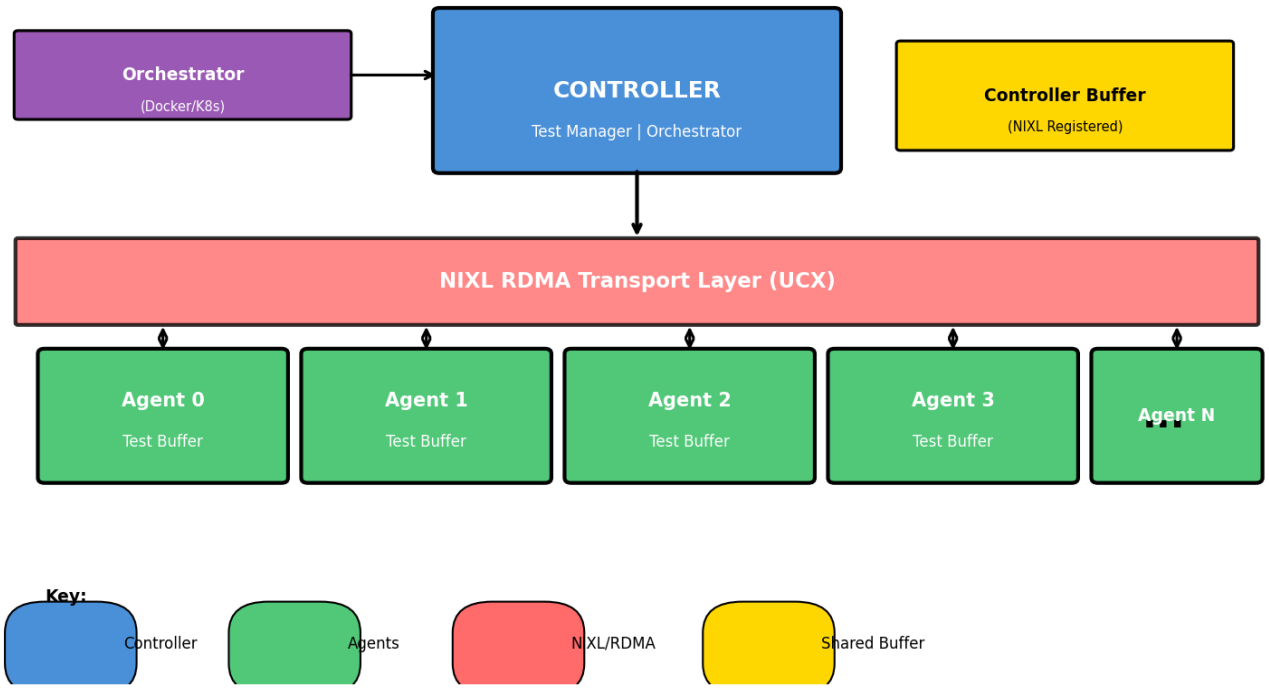
System Design Document

A distributed agent system for measuring memory transfer performance and inferring physical interconnect topology using NVIDIA NIXL framework.

1. System Architecture

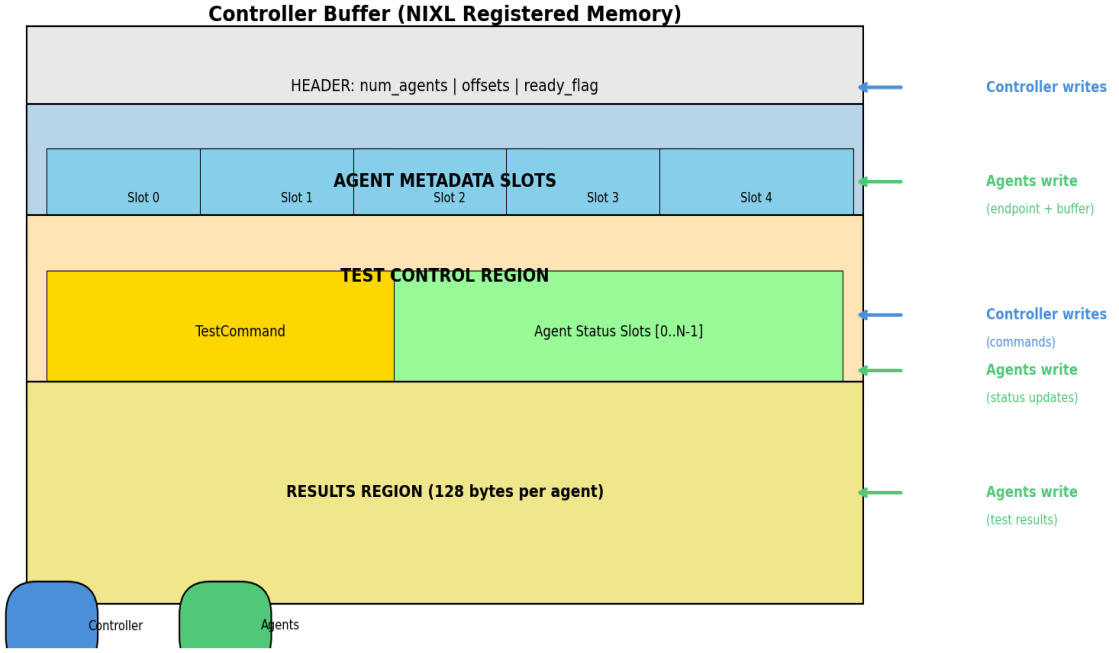
Agents run as containers spawned by the Controller via orchestrator (Docker/K8s). All communication uses NIXL RDMA transfers through a shared controller buffer - no TCP sockets. Each agent registers its NIXL endpoint during bootstrap and receives peer metadata via the controller buffer.

System Architecture - Component Overview



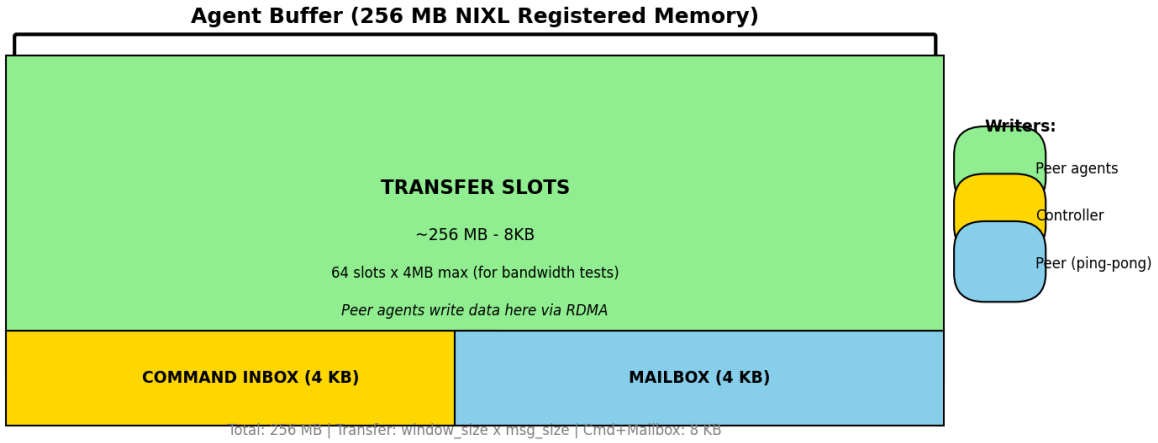
2. Controller Buffer Layout

Single NIXL-registered buffer containing all shared state: agent metadata slots for endpoint blobs, test command region for orchestration, and results region for data collection. NIXL notifications (via genNotif/getNotifs API) are used for async coordination. Agents compute their slot offsets by reading the header.



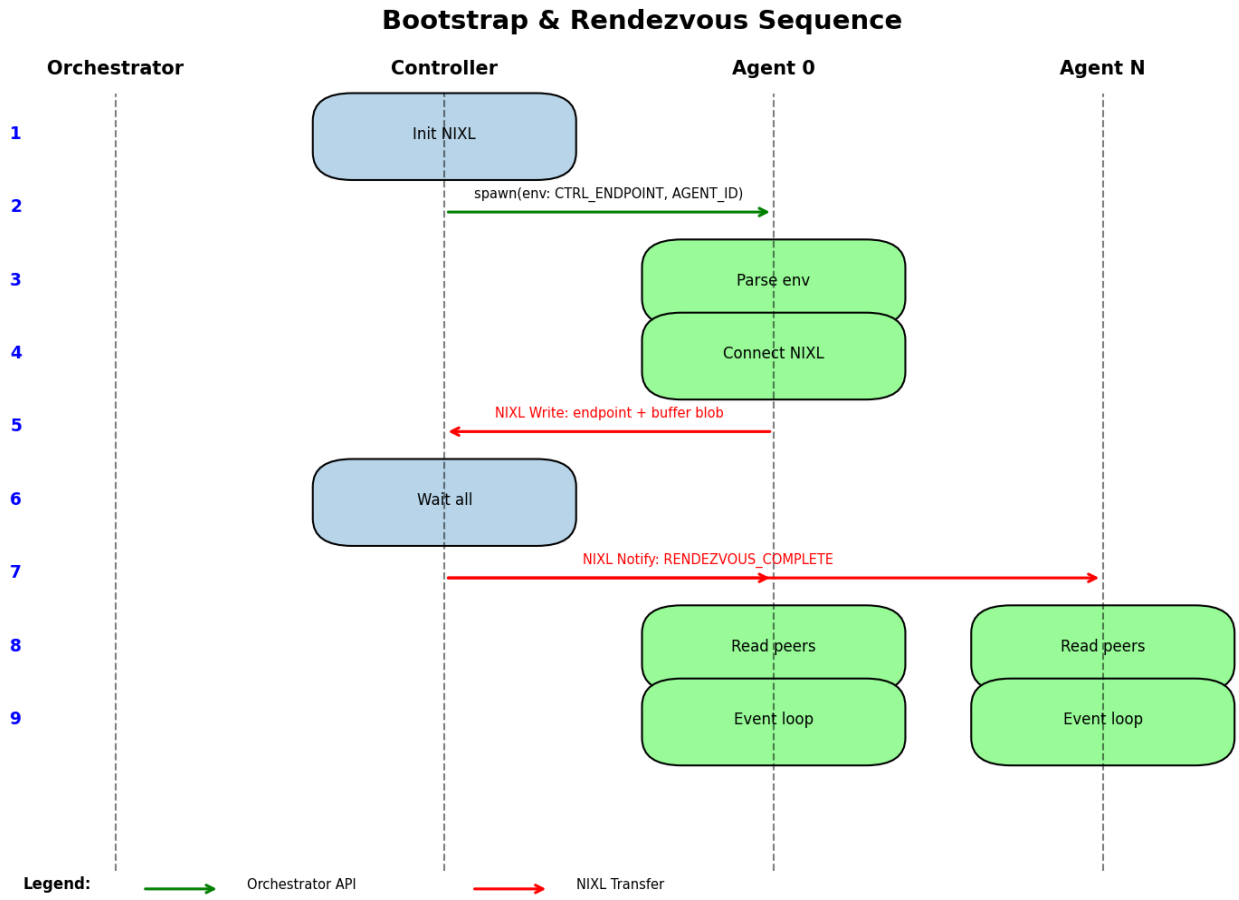
2b. Agent Buffer Layout

Each agent allocates a 256MB NIXL-registered buffer. Transfer slots receive data from peer agents during bandwidth tests. Command inbox receives test commands from controller. Mailbox is used for ping-pong latency measurements.



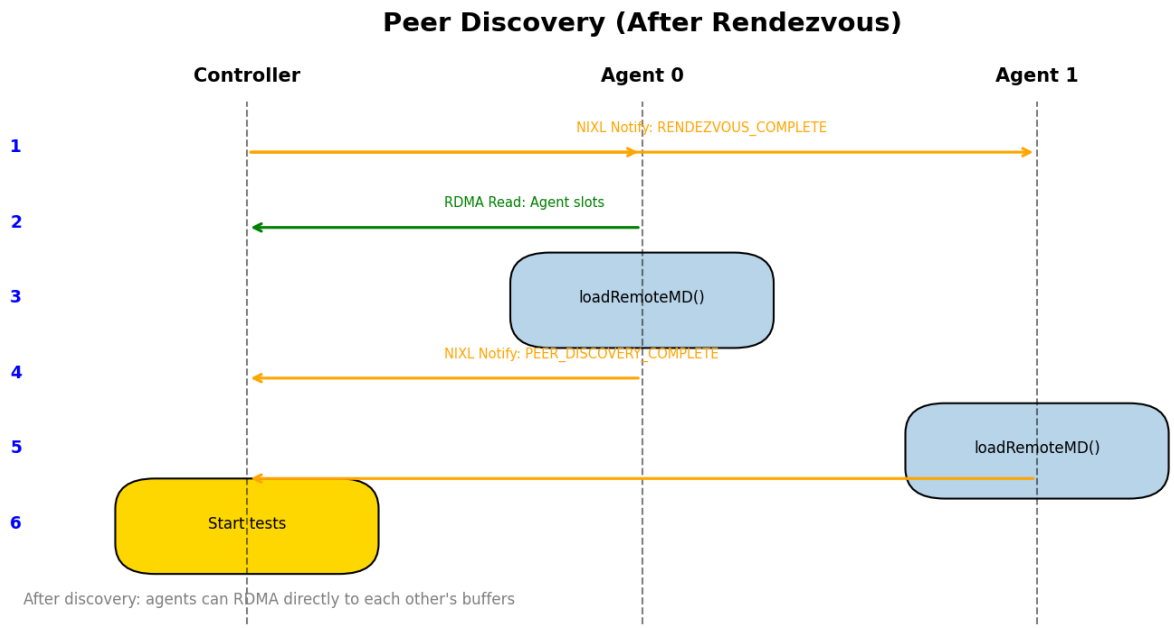
3. Bootstrap & Rendezvous Sequence

Controller spawns agent containers with NIXL metadata via environment variables (CTRL_ENDPOINT, CTRL_BUFFER, AGENT_ID). Agents decode this metadata, establish NIXL connections, and write their own endpoint info. Controller notifies all agents when rendezvous is complete.



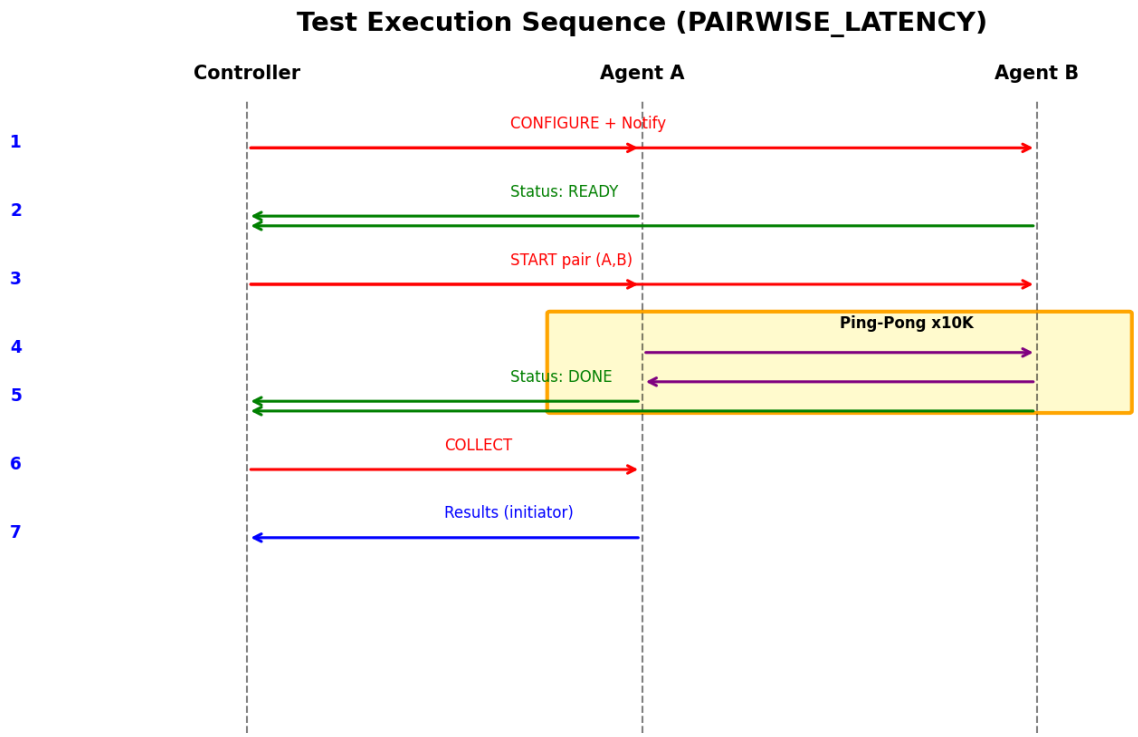
3b. Peer Discovery Sequence

After rendezvous completes, agents read peer metadata from the controller buffer via RDMA, load remote endpoint descriptors into NIXL using loadRemoteMD(), and signal readiness. This enables direct agent-to-agent RDMA communication for performance tests.



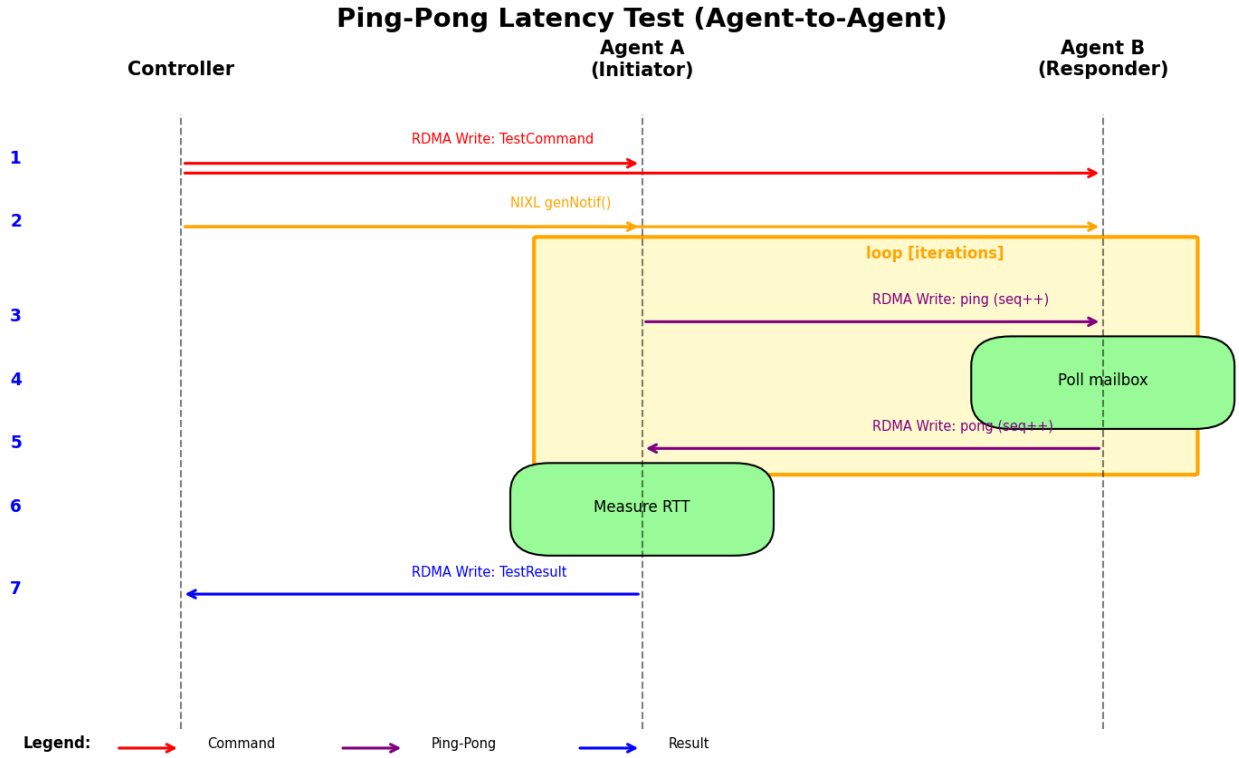
4. Test Execution Protocol

Controller writes test commands to agent command inboxes via RDMA and sends NIXL notifications to wake agents. Agents poll for commands, execute tests, and write results back to the controller's result region. The NIXL notification API (genNotif/getNotifs) provides the async coordination layer.



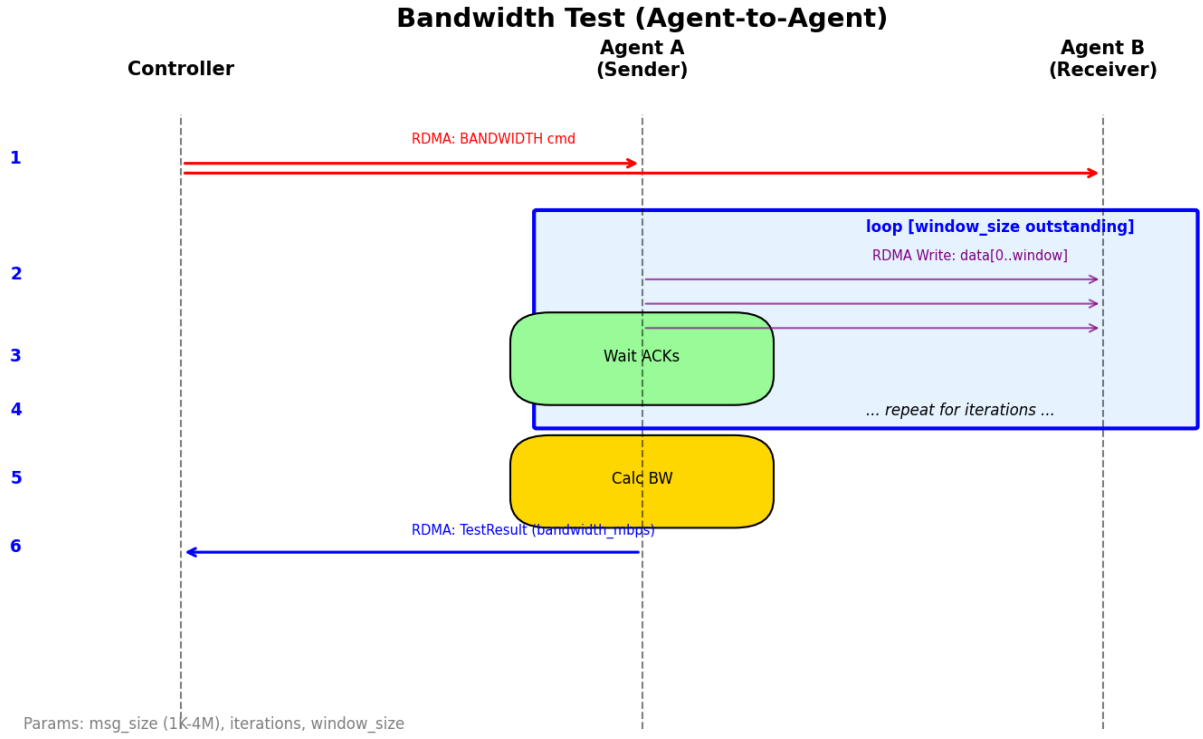
4a. Ping-Pong Latency Test Detail

The initiator agent sends a small message to the responder's mailbox via RDMA write, then polls its own mailbox waiting for a response. The responder polls, detects the incoming message (via sequence number increment), and responds. Round-trip time is measured over thousands of iterations to compute average latency.



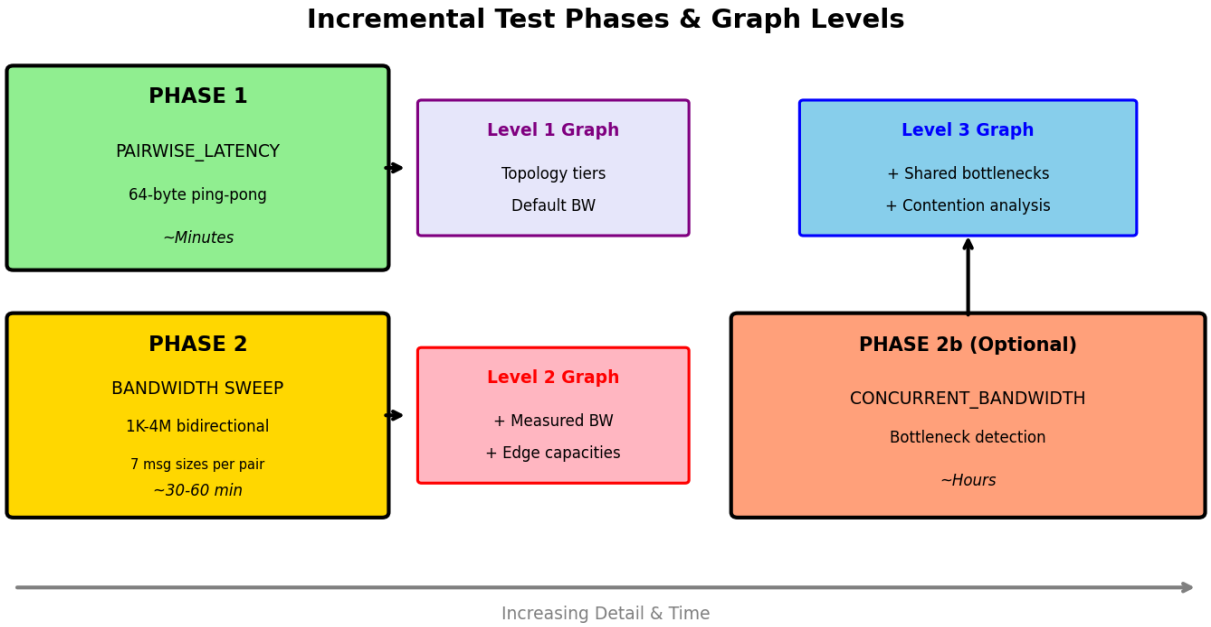
4b. Bandwidth Test Detail

The sender issues a window of concurrent RDMA writes to the receiver's transfer slots, waits for all completions, then repeats for the configured iteration count. Throughput is calculated from total bytes transferred divided by elapsed time. Window size enables pipelining to saturate the link.



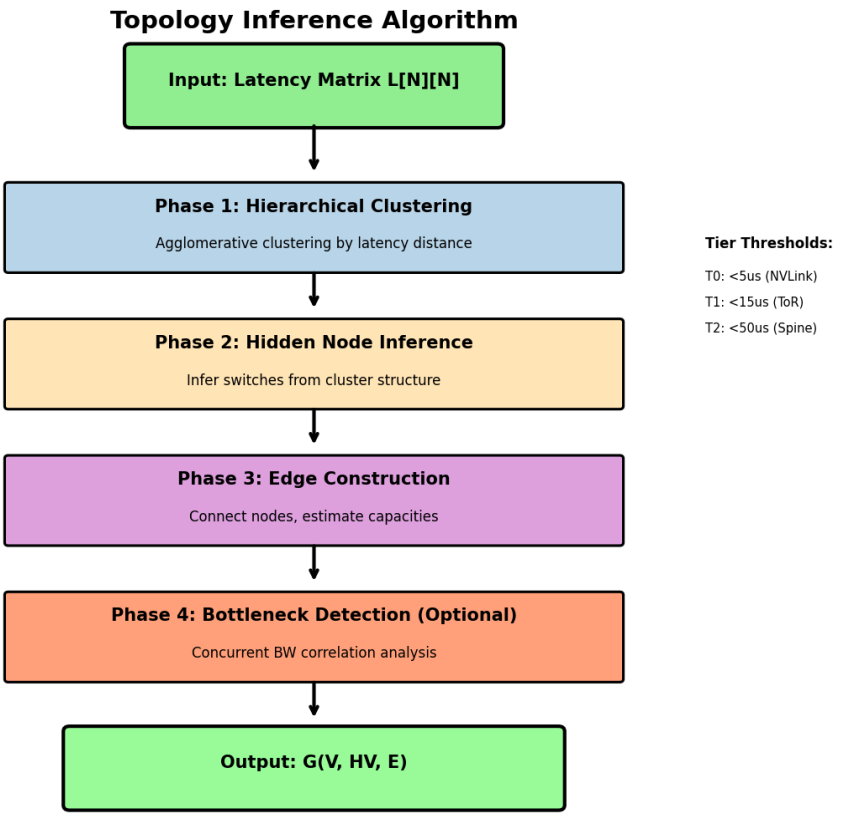
5. Incremental Test Phases

Phase 1 (PAIRWISE_LATENCY) runs quickly with 64-byte ping-pong to produce Level 1 topology with tiers. Phase 2 (BANDWIDTH) sweeps 7 message sizes (1K, 4K, 16K, 64K, 256K, 1M, 4M bytes) bidirectionally per pair, outputting latency_matrix.csv, bandwidth_matrix.csv, and bandwidth_detailed.csv. Optional Phase 2b detects shared bottlenecks via concurrent transfer correlation.



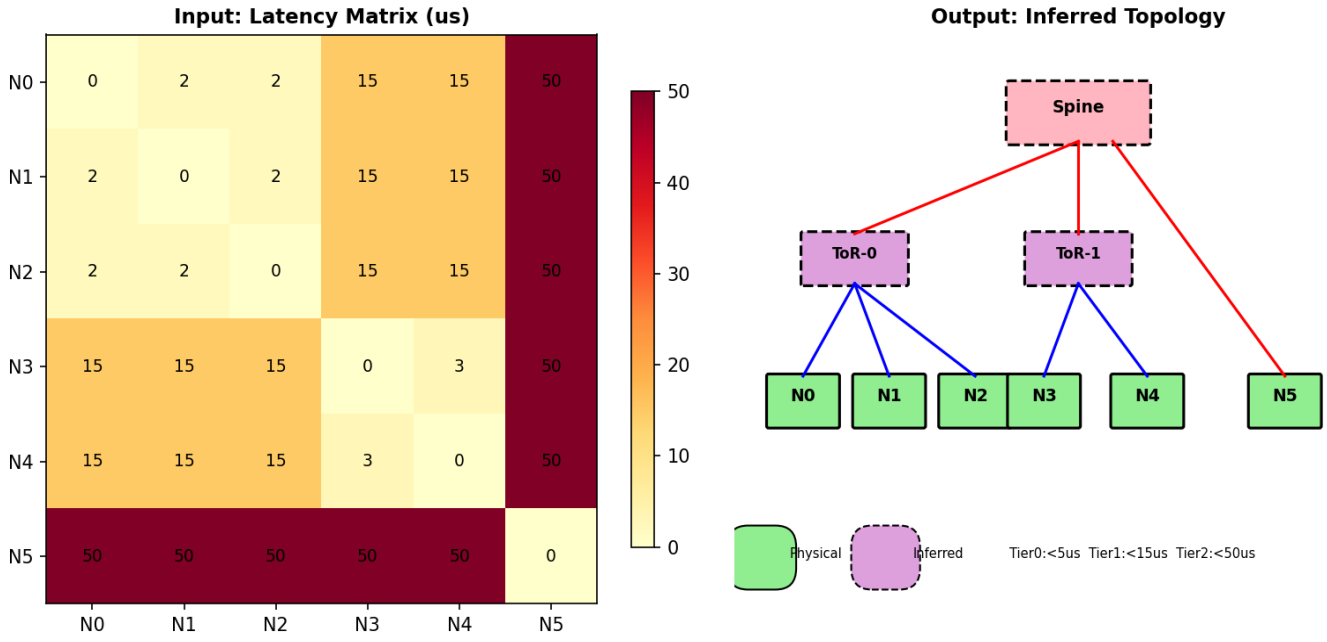
6. Topology Inference Algorithm

Four-phase algorithm: (1) Hierarchical clustering groups nodes by latency similarity, (2) Hidden node inference creates switch nodes for clusters, (3) Edge construction connects physical to hidden nodes with capacity estimates, (4) Optional bottleneck detection identifies shared links.



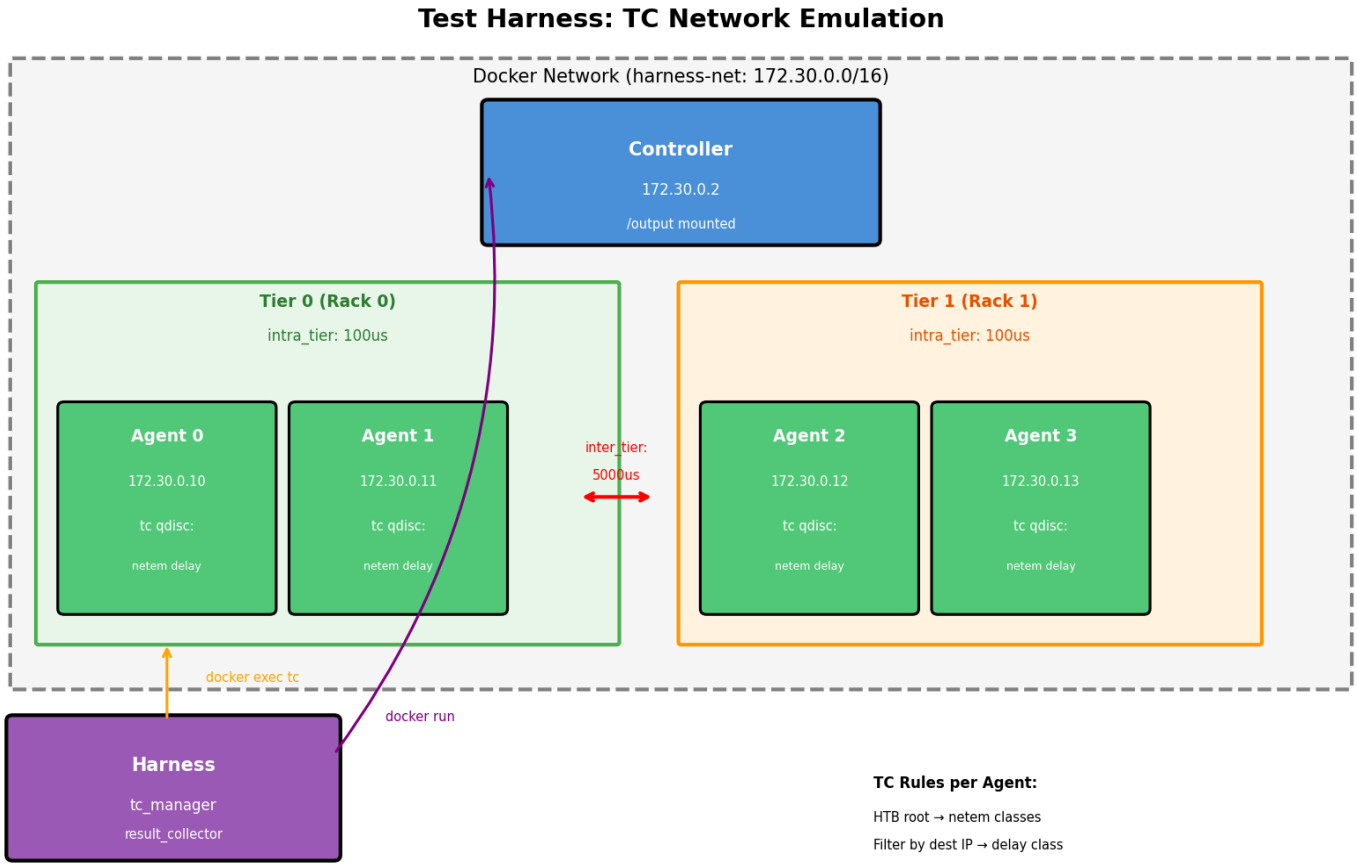
7. Topology Example: Input to Output

Example: 6-node cluster with two tight clusters (N0-N2 at 2us, N3-N4 at 3us) and one isolated node (N5 at 50us). Algorithm infers ToR switches for each cluster and spine switch connecting them, producing hierarchical graph.



8. Test Harness: TC Network Emulation

The test harness creates simulated cluster topologies using Linux tc (traffic control) with netem (network emulation). This enables testing topology discovery algorithms without physical multi-rack infrastructure. The harness manages container lifecycle, applies tc rules to inject latency between tiers, and collects results.



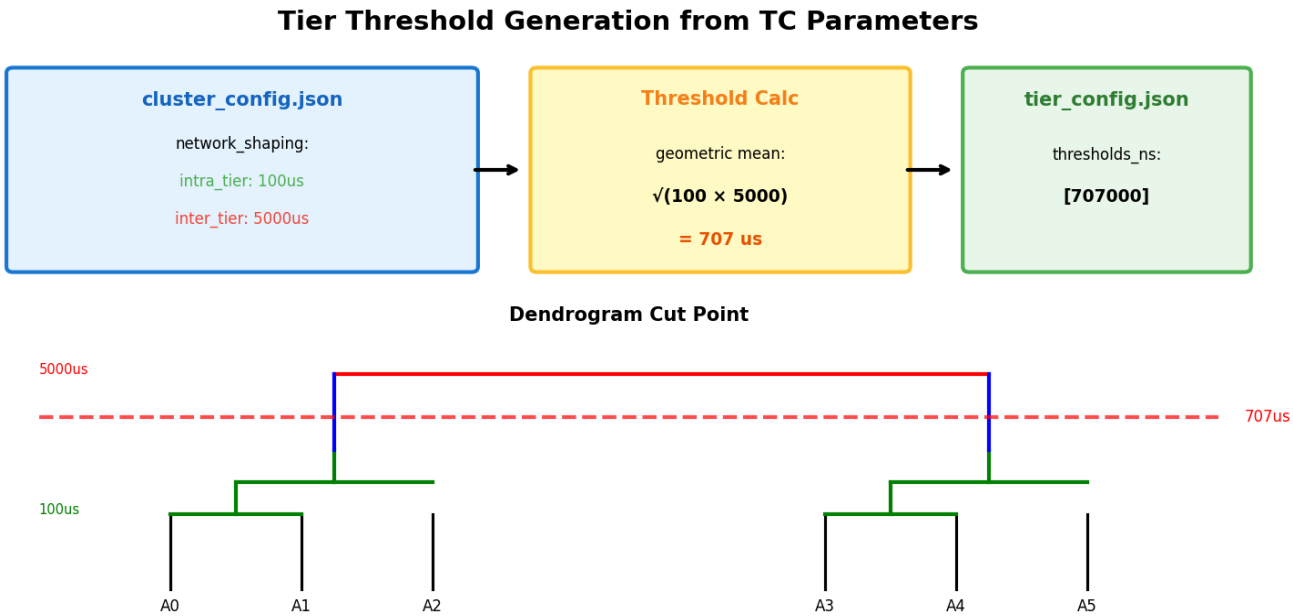
TC Rule Structure: Each agent container gets HTB (Hierarchical Token Bucket) qdisc with netem classes. Traffic is classified by destination IP: intra-tier traffic (same rack) gets low delay (e.g., 100us), inter-tier traffic (cross-rack) gets high delay (e.g., 5000us). Filters route packets to appropriate delay class based on destination.

Configuration Example (two_rack.json):

```
{ "network_shaping": { "tiers": [ { "name": "rack-0", "agents": [0,1], "intra_tier_latency_us": 100 }, { "name": "rack-1", "agents": [2,3], "intra_tier_latency_us": 100 } ], "inter_tier": { "latency_us": 5000, "jitter_us": 500 } } }
```

8b. Tier Threshold Generation

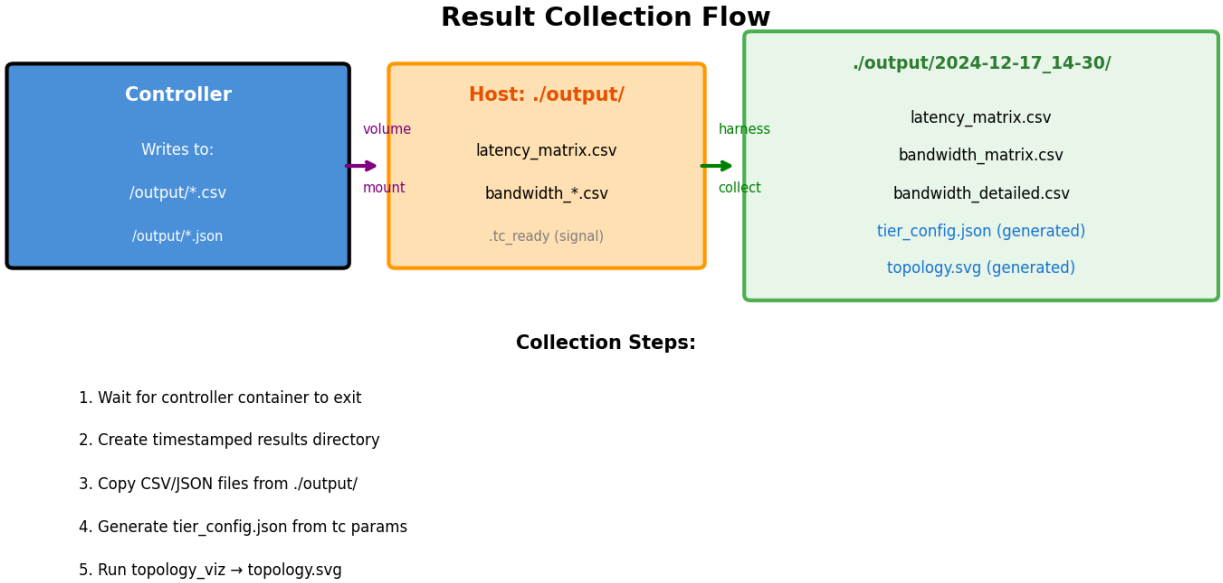
The harness automatically generates tier_config.json for the topology discovery algorithm using the geometric mean of tc latency parameters. This provides optimal log-scale separation for hierarchical clustering - the threshold falls naturally between intra-tier and inter-tier latencies.



Formula: threshold = √(max_intra_tier × inter_tier). For intra=100us and inter=5000us: √(100×5000) = 707us. The generated tier_config.json contains this threshold in nanoseconds, used by topology_viz to draw the dendrogram cut line.

8c. Result Collection

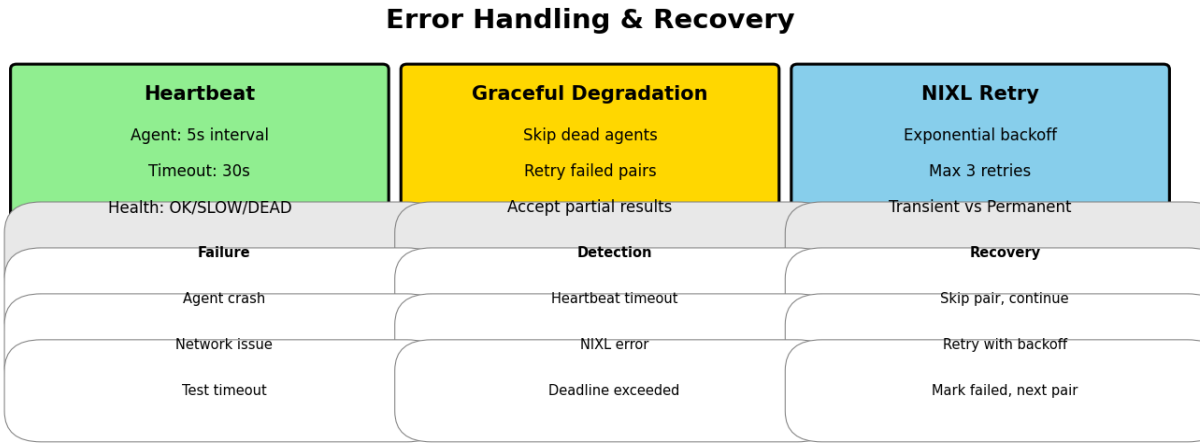
Results flow from controller container to host via Docker volume mount. The harness collect command waits for controller exit, creates timestamped output directory, copies result files, generates tier_config.json from tc params, and optionally runs topology_viz to produce visualization.



Output Files: latency_matrix.csv (NxN RTT in nanoseconds), bandwidth_matrix.csv (peak MB/s per pair), bandwidth_detailed.csv (all message sizes), tier_config.json (auto-generated thresholds), topology.svg (dendrogram visualization).

9. Error Handling & Recovery

Agents maintain heartbeat timestamp; controller detects dead agents via timeout. Failed pairs trigger configurable retry with exponential backoff. Graceful degradation skips unavailable agents and accepts partial results rather than aborting entire test.



10. Implementation Status & Next Steps

Completed:

• Agent bootstrap: env var parsing, NIXL initialization, metadata upload to controller buffer

• Controller buffer management: allocation, header initialization, rendezvous wait loop

• PAIRWISE_LATENCY test: ping-pong protocol, timing measurement, results aggregation

• BANDWIDTH test with message size sweep: 1K, 4K, 16K, 64K, 256K, 1M, 4M bytes

• Bidirectional bandwidth testing (A->B and B->A)

• Container harness with Docker Compose for single-host testing

• CSV outputs: latency_matrix.csv, bandwidth_matrix.csv, bandwidth_detailed.csv

Next Steps:

• LATENCY_SWEEP: Transfer time vs message size curves (8B-64KB, 1000 iterations per size)

• Topology inference: Hierarchical clustering from latency matrix

• Concurrent bandwidth bottleneck detection (optional Phase 2b)

Key Design Decisions Summary

Decision	Choice	Rationale
IPC Mechanism	NIXL-only (no TCP)	Simpler security model, container-friendly
Agent spawn	Orchestrator + env vars	No handshake needed, metadata pre-shared
Coordination	Polling-based	Reliable, works across all environments
Buffer allocation	Upfront (256MB/agent)	Single metadata share, no per-test sync
Bandwidth sweep	7 message sizes (1K-4M)	Characterize full performance curve
CSV output	Matrix + Detailed	Quick topology view + full measurement data
Topology inference	Incremental levels	Quick results first, detail as needed
Results upload	Initiator-only	Reduces data duplication